

RGB Task-Graph History

源码技术手册

Task Graph · 历史特征 · M0-M6 · E2E · Direct Head Fusion · Dynamic Epoch Shuffle

实验范围	A / D / J / M 严格四折 LOSO
随机种子	1 / 2 / 42
训练范围	normal-only / all-runs
任务	35-node分类并聚合为31类Tier3
相机	001484412812
文档版本	2026-07-29

合并来源：CODE_DOCUMENTATION目录下全部11份Markdown文档

目录

RGB Task-Graph History 源码技术手册	9
1. 这套文档解决什么问题	9
2. 推荐阅读顺序	9
3. 文档范围	10
4. 行号与“逐行解析”约定	10
5. 重要结论先览	10
6. 与原有说明/结果报告的关系	10
00 项目、版本与端到端流程	12
1. 研究任务	12
2. 严格实验单位	12
3. Task Graph	12
4. 完整数据流	13
4.1 协议生成	13
4.2 RGB backbone	13
4.3 历史样本	13
5. 模型族	13
5.1 M0	13
5.2 原M1-M3: frozen M0 + history delta	14
5.3 M4-M6: candidate history attention	14
5.4 E2E	14
5.5 Direct Head Fusion	14
6. 35-node到31类Tier3	14
7. 训练与评估	14
01 张量、字段与文件数据字典	16
1. 全局符号	16
2. 索引约定	16
3. Manifest行	16
4. RGB tensor	16
5. RGBClipDataset返回值	17
6. Feature cache	17
7. HistoryExample	17
8. FeatureHistoryDataset单样本	18
9. collate_history_batch	18
10. TaskGraphSpec	18
11. 模型张量	19
M0	19
SingleQueryHistoryModel	19
DirectSingleQueryHistoryModel	19

CandidateHistoryModel	19
12. 概率聚合	19
13. 指标JSON	19
14. Prediction CSV	20
02 graph_history底层包逐文件参考	21
1. graph_history/_init_.py	21
2. constants.py (1–21)	21
2.1 第1行	21
2.2 第3–7行	21
2.3 第9–17行	21
2.4 第19–20行	21
3. utils.py (1–175)	21
seed_everything (14–22)	21
ensure_dir (24–28)	22
ensure_new_output_dir (30–39)	22
read_json / write_json (41–51)	22
read_jsonl (53–65)	22
write_jsonl (67–74)	22
resolve_manifest (76–81)	22
run_key (83–85)	22
select_device (87–91)	22
extract_state_dict (93–105)	22
strip_state_prefixes (107–121)	23
load_compatible_state (123–140)	23
save_checkpoint (142–161)	23
append_csv (163–172)	23
env_or (174–175)	23
4. backbone.py (1–185)	23
get_inplanes (15–17)	23
conv3x3x3 (19–21)	23
conv1x1x1 (23–25)	23
BasicBlock (27–47)	24
Bottleneck (49–72)	24
ResNet.__init__ (75–119)	24
_downsample_basic_block (121–127)	24
_make_layer (129–146)	24
forward_stem (148–150)	24
forward_features (152–160)	24
forward_head (162–165)	25
forward (167–169)	25
generate_model (171–185)	25
5. data.py (1–281)	25
safe_torch_load (18–22)	25

uniform_frame_indices (25–28)	25
RGBVideoTransform (31–69)	25
RGBClipDataset.__init__ (73–101)	25
RGBClipDataset.__getitem__ (106–128)	25
HistoryExample (131–136)	25
load_feature_cache (139–146)	26
FeatureHistoryDataset.__init__ (150–212)	26
feature_dim / __len__ (214–219)	26
__getitem__ (221–248)	26
collate_history_batch (251–280)	26
6. graph.py (1–167)	26
TaskGraphSpec.load (29–89)	26
stable_sample_seed (92–94)	27
randomized_graph_valid_history (97–167)	27
7. protocols.py (1–144)	27
find_fault_manifest (12–22)	27
load_global_fault_run_keys (24–30)	27
_validate_rows (32–52)	27
_sorted (54–63)	27
_summary (65–86)	27
prepare_protocols (88–144)	27
8. metrics.py (1–60)	28
confusion_matrix (9–14)	28
metrics_from_confusion (17–42)	28
classification_metrics (45–48)	28
aggregate_node_probabilities (51–60)	28
9. video_evaluation.py (1–203)	28
_stage_metrics (15–30)	28
_write_predictions (32–41)	28
evaluate_tier3_video_model (43–109)	28
evaluate_node_video_model (111–203)	28

03 模型与训练Engine逐段解析

30

1. models.py	30
1.1 FeatureNodeClassifier (12–25)	30
1.2 freeze_module (27–31)	30
1.3 SingleQueryHistoryModel (33–113)	30
1.4 DirectSingleQueryHistoryModel (116–197)	31
1.5 CandidateHistoryModel (200–371)	32
1.6 build_context_model (374–404)	33
1.7 build_direct_context_model (407–429)	33
2. engine.py	33
2.1 move_batch_to_device (17–21)	33
2.2 forward_node_model (24–34)	34

2.3 compute_loss (37–58)	34
2.4 train_feature_model (61–116)	34
2.5 evaluate_feature_model (120–216)	35

04 tools命令行入口逐文件参考 36

1. 总表	36
2. validate_setup.py (52行)	36
3. prepare_protocols.py (53行)	36
4. guard_output_dir.py (24行)	37
5. train_backbone.py (163行)	37
train_epoch (30–53)	37
evaluate (55–89)	37
main (91–163)	37
6. extract_features.py (108行)	38
7. train_history_model.py (190行)	38
build_loader (30–40)	38
main (42–190)	38
8. train_direct_history_model.py (206行)	39
build_loader (35–45)	39
main (47–206)	39
9. evaluate_e2e_tier3.py (105行)	39
10. train_e2e_node.py (205行)	40
make_loader (31–40)	40
train_epoch (42–64)	40
main (66–205)	40
11. Smoke tests	40
smoke_test_models.py	40
smoke_test_direct_models.py	40
12. summarize_results.py (54行)	41
13. summarize_cross_person.py (182行)	41
小函数	41
collect_rows	41
build_deltas	41
build_aggregate	41
main	41
14. summarize_all_models.py (583行)	41
write_csv (46–53)	41
metric_value (55–60)	41
parse_location (62–99)	41
collect_rows (101–151)	41
collect_stage_rows (153–209)	41
require_complete_grid (211–251)	41
build_pairwise_deltas (253–303)	42
mean_or_none / std_or_none (305–311)	42

aggregate_across_people (313–359)	42
aggregate_stages_across_people (361–372)	42
aggregate_deltas (374–428)	42
build_training_scope_deltas (430–463)	42
aggregate_training_scope_deltas (465–502)	42
main (504–583)	42
15. summarize_direct_head_fusion.py (287行)	42
mean / sample_std / write_csv	42
read_metric (41–53)	42
legacy_model_root (55–60)	42
collect_rows (62–141)	42
build_aggregate (143–224)	43
main (226–287)	43

05 Windows BAT、Shell与Slurm逐脚本参考 44

1. 如何阅读这些脚本	44
2. BAT常见语句	44
3. Windows配置与基础1–12	44
4. 完整all-runs 13–23	45
5. 严格四折多seed 24–30	45
6. Direct Head Fusion 31–33	45
7. Windows组合入口	46
8. Slurm作业脚本 01–12	46
9. Slurm完整all-runs 13–24	47
10. Slurm严格补齐与Direct 25–35	47
11. HPC配置和提交脚本	47
config_hpc.sh	47
单fold提交	48
多fold提交	48
12. Slurm依赖语义	48
13. 不应混用的入口	48

06 Direct Head Fusion实现与新增结果 49

1. 名称	49
2. 研究问题	49
3. 三个模型	49
4. 初始化	49
5. 结果完整性	49
6. test_all主要结果	50
6.1 normal-only	50
6.2 all-runs	50
7. 新结果说明什么	50
7.1 Direct head对带位置模型非常有效	50

7.2 all-runs下M2 Direct略高于M3 Direct	50
7.3 normal-only下两者几乎相同	50
7.4 scope效应不是所有模型都一致	51
8. 当前可支持的结论	51
9. 建议后续分析	51
07 输出、复现与安全边界	52
1. 单participant/seed根目录	52
2. representation与模型目录	52
3. 模型目录文件	52
4. Checkpoint内容	52
5. Feature cache	53
6. 严格汇总目录	53
7. 防覆盖层级	53
Python层	53
脚本层	53
汇总层	53
8. 中断恢复	53
9. 复现检查清单	53
10. 统计解释	54
11. 任务边界	54
12. 部署边界	55
08 配置、资产、依赖与完整文件清单	56
1. configs/experiment_defaults.json	56
顶层任务配置	56
history	56
backbone	56
additional_e2e_baselines	56
complete_all_runs_pipeline	56
recommended_strict_multiseed_extension	56
direct_head_fusion_extension	57
2. assets/integrated_task_graph_latest.json	57
3. assets/integrated_feature_history_matrix.json	57
4. assets/README.md	57
5. requirements.txt	57
6. 原包级说明文件	57
README.md	57
EXPERIMENT_RESULTS_ANALYSIS_2026-07-24.md	58
COMPLETE_EXPERIMENT_CONFIGURATION.md	58
7. Python源码清单	58
graph_history/	58
tools/	59

8. 编排脚本清单	59
9. 结果文件不属于源码	59
10. 当前文档覆盖表	60
Dynamic Epoch Graph-Valid Shuffle实验记录	61
1. 实验目的	61
2. 三个新增模型	61
3. 重排与可复现性	61
4. 实现边界	61
5. 输出隔离与防覆盖	61
6. 运行方式	62

来源文件: README.md

RGB Task-Graph History 源码技术手册

文档版本: 2026-07-29 对应代码包: `graph_history_rgb_cross_person_ADM_2026-07-22` 正式实验: A/D/J/M 四折 LOSO, seed 1/2/42, normal-only 与 all-runs 新增实验: Direct Head Fusion, 以及Dynamic Epoch Graph-Valid Shuffle三模型

1. 这套文档解决什么问题

本目录不是运行说明的简单重复, 而是面向源码阅读、实验复现和后续开发的技术手册。文档把研究设计、数据字段、张量维度、Python实现、Windows/HPC编排和结果文件连接成一条完整链路。

阅读完后应能够回答:

- 一个RGB clip如何变成[512]维冻结特征;
- 同run历史如何从manifest构造、排序、padding并送入attention;
- M0-M6分别使用哪些输入, 哪些权重被冻结, 最终logits如何产生;
- 35-node概率如何聚合成31类Tier3概率;
- Direct Head Fusion为何不加载M0, 以及它与原delta head的本质区别;
- 一个fold、一个seed、一个scope由哪些脚本按什么顺序完成;
- 每一种checkpoint、cache、metrics、prediction和summary写到哪里;
- 哪些统计是participant级聚合, 哪些只是seed-fold或run级描述。

2. 推荐阅读顺序

1. 00_project_and_pipeline.md

先理解研究目标、版本范围、模型族和端到端数据流。

2. 01_tensor_and_data_dictionary.md

建立统一符号, 理解manifest、feature cache、batch和输出文件的内容。

3. 02_core_package_reference.md

从`constants.py`、`utils.py`、`backbone.py`、`data.py`、`graph.py`开始阅读底层实现。

4. 03_models_and_engine_line_by_line.md

逐段理解M0-M6、Direct Head Fusion、loss、训练和评估。

5. 04_tools_reference.md

阅读所有Python命令行入口及其输入输出。

6. 05_bat_slurm_reference.md

理解Windows BAT、HPC Slurm作业和提交脚本如何编排实验。

7. 06_direct_head_fusion_results.md

查看新增Direct Head Fusion完整结果及与M0/原M1-M3的严格配对比较。

8. 07_outputs_reproducibility_and_safety.md

查看输出树、防覆盖、`completed.json`和结果解释边界。

9. 08_configuration_assets_and_inventory.md

查看配置、依赖、资产、原有说明文件和完整源码清单。

10. 09_dynamic_epoch_shuffle.md

查看三种dynamic模型、逐epoch合法重排、初始化边界、输出隔离和运行入口。

3. 文档范围

正文以当前正式实验包为唯一实现依据：

```
D:\Junxi_data\Objective3_thermal_crimp\codex_and_files\
graph_history_rgb_cross_person_ADM_2026-07-22
```

旧graph_history_rgb_experiments_2026-07-20只属于先导实验历史，不作为当前四折代码的实现依据。

覆盖内容：

- graph_history/：14个Python模块；
- tools/：18个Python命令行入口；
- bat/：54个Windows入口与组合脚本；
- slurm/：38个作业脚本、17个配置/提交shell脚本；
- assets/与configs/；
- M0-M6、三个E2E对照、三个Direct Head Fusion模型和三个Dynamic模型；
- normal-only、all-runs、A/D/J/M、seed 1/2/42和三个测试split；
- 结果、预测、概率、汇总和防覆盖机制。

不把outputs/中的二进制checkpoint逐参数展开；文档解释其schema、来源和用途。统计数值只取自实际 CSV/JSON，不从旧报告手工复制。

4. 行号与“逐行解析”约定

源码行号以2026-07-29检查到的当前文件为基准。后续修改代码后行号可能移动，因此文档同时使用“函数名 + 语义块 + 当前行号”定位。

逐行解析遵循以下规则：

- import、赋值、条件、循环、张量运算、I/O、返回值均解释；
- 多行函数调用按一个语义操作解释，同时说明每个实参；
- 空行、纯括号行和只用于排版的换行不单独重复解释；
- 动态列表和字典写“内容schema”，张量写shape/dtype/device；
- shape中的B、L等符号在张量字典中统一定义。

5. 重要结论先览

- 当前最终任务是预切分RGB clip分类，不是连续视频检测或fault detection。
- M0-M6预测35个graph node；Tier3指标来自35-node概率求和，不是argmax node再映射。
- M5读取真实历史node，是oracle实验；M6使用冻结M0概率，是可部署soft-relation版本。
- M3使用graph-valid重排，但重排不读取当前目标node，因此避免current-label leakage。
- Direct Head Fusion不加载M0；attention、fusion和随机初始化35-node head联合训练50 epoch。
- Dynamic Frozen-M0 Delta加载并冻结M0；Dynamic Joint-Head Delta明确不加载M0，node head与delta联合训练。
- Dynamic Direct Fusion不使用delta；三个dynamic模型只在训练期逐epoch重排，主测试使用固定合法顺序。
- 新Direct结果中，m2_direct与m3_direct大幅高于原delta版本；完整结果见第6卷。

6. 与原有说明/结果报告的关系

原包中的三份说明仍保留并应共同阅读：

README.md
EXPERIMENT_RESULTS_ANALYSIS_2026-07-24.md
COMPLETE_EXPERIMENT_CONFIGURATION.md

本技术手册侧重“代码如何工作”；实验报告侧重“结果说明什么”；原README侧重“如何运行”。当三者 出现版本差异时，先检查当前源码、实际CSV/JSON与文件更新时间，不直接复制旧结论。

来源文件: 00_project_and_pipeline.md

00 项目、版本与端到端流程

1. 研究任务

输入是单相机001484412812采集的RGB视频clip。基础视觉任务有两种标签空间：

- 35个Task Graph node：表达动作位于工艺流程中的具体位置；
- 31个Tier3动作：若不同node具有相同动作语义，会聚合到同一个Tier3类别。

因此，35-node任务比31类动作识别多解决一层“流程位置消歧”。例如Stage 2中两次press pedal 视觉外观相同，但属于不同node；仅看当前clip容易互相混淆，而同run历史可提示当前处于第一次还是 第二次压踏板。

2. 严格实验单位

正式网格：

```
participant = A, D, J, M
seed        = 1, 2, 42
scope       = normal_only, all_runs
models      = M0-M6 + 3 E2E + 3 Direct扩展
test split  = test_normal, test_fault, test_all
```

每个fold只用另外三位participant训练。held-out participant不进入训练、validation或checkpoint 选择。没有early stopping，统一使用最后epoch的last.pth。

normal-only和all-runs是两个完整、独立的representation/training scope：

- normal_only backbone仅用训练participants的正常runs；
- all_runs backbone使用训练participants的正常与fault runs；
- 两种scope各自提取feature cache并训练下游模型；
- 严格scope差值使用相同participant、seed、model和split配对。

3. Task Graph

Task Graph资产：

```
assets/integrated_task_graph_latest.json
assets/integrated_feature_history_matrix.json
```

每个可预测node至少给出：

- node_idx: JSON使用1到35；模型内部转换为0到34；
- stage_id: 1、2或3；
- action_id_tier3: 0到30；
- 必须前序node；
- 可选前序node；
- 必须立即前序node。

relation matrix方向固定：

```
row    = 当前候选node v
column = 历史node u
```

代码将I/M/O/X/S映射为整数0/1/2/3/4。JSON中的. 被规范化为X。固定relation matrix 本身不训练；M5/M6学习的是每个attention head对五类relation的标量bias。

4. 完整数据流

4.1 协议生成

原始manifest
 → 按held-out participant划分train/test
 → 用fault manifests识别完整fault run
 → normal_only/train.jsonl
 → all_runs/train.jsonl
 → test_normal.jsonl
 → test_fault.jsonl
 → test_all.jsonl

历史始终只来自相同participant + run且annotation_row_index更小的clip。没有未来信息，没有跨run memory。

4.2 RGB backbone

RGB tensor [原始帧数, 3, H0, W0]
 → 均匀选择16帧
 → resize/crop、颜色增强和归一化
 → 转为[3, 16, 224, 224]
 → batch后[B, 3, 16, 224, 224]
 → ResNet3D-18
 → 31类Tier3 logits [B, 31]

训练后保存last.pth。特征提取阶段读取分类头之前的512维向量：

```
feature [B, 512]
tier3_logits [B, 31]
```

并连同逐样本record与metadata保存到.pt feature cache。

4.3 历史样本

FeatureHistoryDataset先按participant + run分组，再按annotation_row_index排序。对于run中位置t的当前clip，历史是[0:t]。

- M1/M2/M4/M5/M6与Direct M1/M2使用实际发生顺序；
- M3与Direct M3使用确定性的graph-valid随机拓扑序；
- graph-valid排序只依据已经观察到的历史node之间的关系；
- 当前目标node不传给排序函数，防止标签泄漏；
- 历史中node重复时回退实际顺序。

batch collate把变长历史padding到当前batch最大长度L：

```
current_feature      [B, 512]
history_features     [B, L, 512]
history_position_ids [B, L]
history_node_classes [B, L]
history_padding_mask [B, L]
node_target         [B]
tier3_target        [B]
```

padding mask中True表示padding、不可被attention读取；有效历史为False。

5. 模型族

5.1 M0

```
LayerNorm(512)
→ Dropout
→ Linear(512, 35)
```

只读取当前clip特征，训练35-node分类头。

5.2 原M1–M3: frozen M0 + history delta

```
baseline_logits = frozen_M0(current_feature) [B, 35]
context = single-query attention(current, history) [B, 256]
delta = MLP([current_projected, context]) [B, 35]
logits = baseline_logits + sigmoid(scale_logit)*delta [B, 35]
```

- M1: 无位置embedding;
- M2: 实际历史 + 位置embedding;
- M3: graph-valid历史 + 位置embedding。

覆盖训练模式时，baseline始终保持`eval()`，且其参数`requires_grad=False`。

5.3 M4–M6: candidate history attention

35个候选node分别形成query，因此attention张量保留candidate维：

```
queries [B, heads, 35, head_dim]
keys/values [B, heads, L+1, head_dim]
scores [B, heads, 35, L+1]
context [B, 35, 256]
delta [B, 35]
```

额外的+1是可学习null-history token，保证空历史样本仍有合法attention key。

- M4: graph source=`none`，bias全零；
- M5: 历史真实node one-hot概率，oracle；
- M6: 冻结M0对历史feature的35-node softmax概率。

5.4 E2E

- E2E-Tier3-Scratch: RGB直接预测31类，只报告Tier3。
- E2E-Node-Scratch: ResNet3D从scratch预测35-node。
- E2E-Node-From-Tier3: 加载Tier3 backbone兼容参数，跳过形状不同的最终fc，再训练35-node。

5.5 Direct Head Fusion

```
current_feature [B, 512]
history context [B, 256]
concat [B, 768]
Linear(768, 512) + LayerNorm
node head Linear(512, 35)
```

它不读取M0 checkpoint，不做logit residual。fusion层初始权重为`[I512, 0]`，bias为0，因此训练开始时`fused_feature == current_feature`；随后attention、fusion和node head联合学习。

6. 35-node到31类Tier3

设`p_node[... , n]`为node概率，`map[n]`为该node对应的Tier3类别，则：

$p_{\text{tier3}}[... , c] = \sum p_{\text{node}}[... , n]$ ，对所有`map[n] == c`的node求和

代码使用`scatter_add`实现，因此保留完整概率质量。它不是先取最大概率node再做标签映射。

7. 训练与评估

history/direct模型默认主loss：

```
L_node = CrossEntropy(node_logits, node_target)
```

可选Tier3辅助loss从聚合概率中取真实Tier3概率的负对数：

```
L = L_node + action_loss_weight * L_tier3
```

当前默认`action_loss_weight=0.0`，即只优化node交叉熵。

评估同时生成：

- 35-node accuracy、macro-F1、balanced accuracy;
- 31类Tier3相同指标;
- Stage 1/2/3分阶段指标;
- 逐样本预测与置信度;
- node probability tensor。

macro-F1与balanced accuracy只在当前split中真实出现的类别上平均，因此比较不同participant的 fault split时必须同时检查 `present_class_count`。

来源文件: 01_tensor_and_data_dictionary.md

01 张量、字段与文件数据字典

1. 全局符号

符号	含义	默认/范围
B	batch size	backbone默认16；history默认64
T	每个clip选取的帧数	16
L	当前batch padding后的最大历史长度	0到35
F	冻结RGB feature维度	512
D	history attention内部维度	256
N	Task Graph node数	35
C	Tier3类别数	31
A	attention head数	4
Dh	每个head维度	D/A=64
S	某个split的样本数	随participant/split变化

2. 索引约定

概念	文件/报告	模型内部
graph node	node_idx=1..35	node_target=0..34
Tier3	tier3_id=0..30	不变, 0..30
stage	1..3	不变, 1..3

写prediction CSV时, node预测会加1恢复为1..35; Tier3不加1。

3. Manifest行

每一行是一个JSON对象。代码直接依赖的字段:

字段	类型	内容
sample_name	string	clip唯一标识, 也是cache查找键
participant	string	A/D/J/M
run	string	participant内部run标识
annotation_row_index	int	run内时序位置; 更小表示更早
node_idx	int	1到35
tier3_id	int	0到30
stage_id	int	1、2或3
<camera_id>_rgb	string	相对dataset root的RGB tensor路径

run_key(row) 返回 (participant, run), 用于严格限制历史来源。

4. RGB tensor

磁盘对象可以是tensor本身, 也可以是包含frames的字典:

原始 video: [T_raw, 3, H_raw, W_raw]

约束:

- 必须是PyTorch tensor;
- 必须4维;
- 第二维必须为3;
- `T_raw > 0`。

处理后单样本:

```
均匀采样      [16, 3, H_raw, W_raw]
transform后    [16, 3, 224, 224]
permute后     [3, 16, 224, 224]
```

Dataloader默认collate后:

```
video [B, 3, 16, 224, 224], float32
```

训练transform包含随机裁剪、ColorJitter、随机灰度; 水平和垂直翻转概率均显式设为0。验证/测试只resize、转float并归一化。

5. RGBClipDataset返回值

key	单样本类型/shape	batch后
video	float tensor <code>[3, T, 224, 224]</code>	<code>[B, 3, T, 224, 224]</code>
tier3_target	Python int	long tensor <code>[B]</code>
node_target	Python int, 0-based	long tensor <code>[B]</code>
stage_id	Python int	long tensor <code>[B]</code>
sample_name	string	<code>list[str]</code> , 长度B
participant	string	<code>list[str]</code>
run	string	<code>list[str]</code>
annotation_row_index	int	默认collate为tensor <code>[B]</code>

6. Feature cache

顶层必须是字典, 并至少包含:

key	内容
features	<code>[S, 512]</code> , 分类头之前的冻结RGB特征
tier3_logits	<code>[S, 31]</code>
records	长度S的manifest行/样本metadata
metadata	checkpoint、相机、提取配置等来源信息

`load_feature_cache`检查`len(records) == features.shape[0]`, 但不会自动修复错位。`sample_name`是selection manifest与cache对齐的主键。

7. HistoryExample

这是冻结dataclass, 不包含复制后的feature, 只保存索引和行对象:

字段	类型	内容
current_cache_index	int	当前clip在cache第一维的行号
history_cache_indices	<code>tuple[int,...]</code>	历史clips在cache中的行号
current_row	dict	当前manifest行
history_rows	<code>tuple[dict,...]</code>	以实际或graph-valid顺序排列的历史行

这样多个Dataset样本可以共享同一个`features [S, 512]`，避免为每个样本复制历史特征。

8. FeatureHistoryDataset单样本

若当前历史长度为`L`：

key	shape/类型	内容
<code>current_feature</code>	<code>[512]</code> , float32	当前clip冻结特征
<code>history_features</code>	<code>[1, 512]</code> , float32	有序历史特征
<code>history_position_ids</code>	<code>[1]</code> , long	<code>1, 1-1, ..., 1</code> , 1代表最近
<code>history_node_classes</code>	<code>[1]</code> , long	历史真实node, 0-based
<code>node_target</code>	int	当前真实node, 0-based
<code>tier3_target</code>	int	当前真实Tier3
<code>stage_id</code>	int	当前stage
<code>history_sample_names</code>	<code>list[str]</code>	与历史特征顺序完全一致

空历史：

```
history_features      [0, 512]
history_position_ids  [0]
history_node_classes  [0]
```

9. collate_history_batch

设batch内最大历史长度为`L`：

key	shape/dtype	padding规则
<code>current_feature</code>	<code>[B, 512]</code> float32	stack
<code>history_features</code>	<code>[B, L, 512]</code> float32	0
<code>history_position_ids</code>	<code>[B, L]</code> long	0
<code>history_node_classes</code>	<code>[B, L]</code> long	-1
<code>history_padding_mask</code>	<code>[B, L]</code> bool	padding=True, 有效=False
<code>node_target</code>	<code>[B]</code> long	无
<code>tier3_target</code>	<code>[B]</code> long	无
<code>stage_id</code>	<code>[B]</code> long	无

如果整个batch都是run的第一个clip, `L=0`，张量shape合法为`[B, 0, ...]`。模型额外添加null token，因此attention仍能运行。

10. TaskGraphSpec

字段	shape/类型	内容	
<code>relation_ids</code>	<code>[35, 35]</code> long	<code>[候选当前node, 历史node]</code> 关系ID	
<code>node_to_tier3</code>	<code>[35]</code> long	0到30	
<code>node_to_stage</code>	<code>[35]</code> long	1到3	
<code>all_must_previous</code>	<code>dict[int, tuple[int, ...]]</code>	使用1-based node; M3排序实际使用	
<code>immediate_previous</code>	<code>`dict[int, int</code>	<code>None]</code>	使用1-based node; 当前M3排序未直接使用
<code>atomic_sequences</code>	<code>tuple of tuples</code>	不可拆分的局部顺序	

11. 模型张量

M0

input features	[B, 512]
normalized	[B, 512]
logits	[B, 35]
probabilities	[B, 35]

SingleQueryHistoryModel

current projection	[B, 256]
history projection	[B, L, 256]
null token	[B, 1, 256]
history with null	[B, L+1, 256]
query	[B, 1, 256]
attention output	[B, 1, 256]
context after squeeze	[B, 256]
concat current/context	[B, 512]
history delta	[B, 35]
baseline logits	[B, 35]
final logits	[B, 35]

DirectSingleQueryHistoryModel

current feature	[B, 512]
attention context	[B, 256]
concat	[B, 768]
fusion linear output	[B, 512], 诊断字典中的fused_feature
node classifier LN	[B, 512]
node classifier Linear	[B, 35]

CandidateHistoryModel

candidate embedding	[35, 256]
expanded candidates	[B, 35, 256]
queries	[B, 4, 35, 64]
keys/values	[B, 4, L+1, 64]
attention scores	[B, 4, 35, L+1]
history node probs	[B, L, 35]
graph bias	[B, 4, 35, L]
context	[B, 35, 256]
delta	[B, 35]

M5的history node probabilities来自one-hot真实标签；padding位置归零。M6由冻结M0输出softmax；M4即使计算路径需要兼容shape，其graph bias返回全零。

12. 概率聚合

输入允许任意前导维度：

node_probabilities	[..., 35]
node_to_tier3	[35]
result	[..., 31]

例如普通batch为[B, 35] → [B, 31]。`scatter_add`确保映射到同一Tier3的多个node概率相加。

13. 指标JSON

顶层：

```
split
samples
node
tier3
per_stage
```

`node`和`tier3`均包含：

```
accuracy
macro_f1
balanced_accuracy
present_class_count
```

```
total_class_count
per_class_precision
per_class_recall
per_class_f1
support
confusion_matrix
```

Node confusion matrix为[35, 35], Tier3为[31, 31]。行是真实类别, 列是预测类别。

14. Prediction CSV

历史模型评估至少写:

```
sample_name
participant
run
annotation_row_index
stage_id
true_node_idx
pred_node_idx
true_tier3_id
pred_tier3_id
node_confidence
tier3_confidence
```

true_node_idx/pred_node_idx恢复为1-based; 概率文件中的35维列仍按内部0-based位置排列。

来源文件: 02_core_package_reference.md

02 graph_history底层包逐文件参考

本卷按“无状态工具 → 视频backbone → 数据集 → 图结构 → 协议 → 指标 → 评估”的顺序解释。模型和训练engine放在下一卷。

1. graph_history/__init__.py

包初始化文件只暴露包版本/基本标识，不执行训练、不读数据、不写文件。它的主要作用是让 `graph_history` 成为可导入 Python 包。

2. constants.py (1–21)

2.1 第1行

`from __future__ import annotations` 延迟类型注解求值，使 `Path` | `str` 等注解不会在导入阶段产生不必要的运行时依赖。

2.2 第3–7行

变量	内容
<code>NUM_GRAPH_NODES</code>	35
<code>NUM_TIER3_CLASSES</code>	31
<code>DEFAULT_CAMERA_ID</code>	"001484412812"
<code>DEFAULT_RGB_MEAN</code>	三通道均值
<code>DEFAULT_RGB_STD</code>	三通道标准差

`mean/std` 是长度3的tuple，顺序对应RGB通道，用于 `Normalize`。

2.3 第9–17行

`MODEL_NAMES` 把实验短名映射为描述名。它是命名/校验表，不负责实例化模型：

```
m0 current_only
m1 history_no_position
m2 actual_history
m3 graph_valid_shuffle
m4 candidate_no_graph
m5 graph_oracle
m6 graph_predicted
```

2.4 第19–20行

`RELATION_TO_ID` 固定 I/M/O/X/S → 0/1/2/3/4。`ID_TO_RELATION` 用字典推导反转映射。ID 是 relation embedding/bias 索引，不代表强弱顺序。

3. utils.py (1–175)

seed_everything (14–22)

- 将输入转为 int；
- 设置 Python `random`、NumPy 和 PyTorch CPU 随机种子；
- 若 CUDA 可用，对所有 GPU 设置 seed；
- 配置 cuDNN deterministic/benchmark 状态，降低重复运行差异。

它控制模型初始化、DataLoader 相关随机性和 graph-valid 排序的基础 seed，但不能保证不同硬件/算子 100% 位级一致。

ensure_dir (24–28)

把输入转成Path，以`parents=True`，`exist_ok=True`创建目录并返回Path。允许目录已存在，适合普通 日志/测试结果子目录，不提供防覆盖。

ensure_new_output_dir (30–39)

这是实验安全核心：

- 目标不存在：创建；
- 目标存在但为空：可继续；
- 目标非空且`overwrite=False`：抛出异常；
- 只有显式`overwrite=True`才允许复用非空目录。

标准BAT/Slurm入口不传`--overwrite`。

read_json / write_json (41–51)

- UTF-8读取/写入；
- `write_json`先确保父目录存在；
- 使用缩进和非ASCII保留，便于中文/路径人工检查。

read_jsonl (53–65)

逐行读取JSONL：

- 去除首尾空白；
- 跳过空行；
- 每个非空行独立`json.loads`；
- 返回`list[dict]`。

输入manifest通常只有几千行，因此当前实现整体载入内存。

write_jsonl (67–74)

逐个序列化row并写换行。`rows`接受任意Iterable，因此可传list或生成器。

resolve_manifest (76–81)

若manifest是绝对路径直接返回；否则拼到dataset root。该函数只解析路径，不检查文件一定存在。

run_key (83–85)

返回：

```
(str(row["participant"]), str(row["run"]))
```

这是历史分组与fault run识别的关键复合键。

select_device (87–91)

- `requested != "auto"`：直接构造`torch.device(requested)`；
- `auto`：CUDA可用则`cuda`，否则`cpu`。

extract_state_dict (93–105)

兼容不同checkpoint包装：

- checkpoint本身可能就是state dict；
- 也可能在`state_dict`、`model_state`或`model`字段；
- 返回值必须是`dict[str, Tensor]`；

- 无可识别结构时抛出错误。

`strip_state_prefixes` (107–121)

规范checkpoint key, 移除常见包装前缀, 例如DataParallel产生的`module.`。值tensor不复制, 只重建 键字典。

`load_compatible_state` (123–140)

步骤:

1. CPU加载checkpoint;
2. 提取并清理state dict;
3. 与目标模型当前state逐key比较;
4. 只保留“键存在且shape完全相同”的tensor;
5. `strict=False`加载;
6. 返回已加载、缺失、意外和shape不兼容键的报告。

Tier3迁移到35-node时, 最终`fc.weight/fc.bias`因输出维度31→35而自动跳过。

`save_checkpoint` (142–161)

保存模型、optimizer、epoch、配置和可选extra。模型tensor由`state_dict()`产生; 若需要断点续训, optimizer state也一并保留。当前正式协议仍以最后epoch完成后保存的`last.pth`为最终模型。

`append_csv` (163–172)

追加单行CSV:

- 文件不存在时写header;
- 存在时只追加row;
- 字段顺序取当前row的keys;
- 适合训练摘要, 不用于需要原子替换的完整汇总。

`env_or` (174–175)

读取环境变量; 不存在时返回default。用于BAT/Slurm覆盖默认路径和超参数。

4. `backbone.py` (1–185)

实现3D ResNet, 输入维度顺序为 `[B, C, T, H, W]`。

`get_inplanes` (15–17)

返回 `[64, 128, 256, 512]`, 对应四个ResNet stage的输出channel。

`conv3x3x3` (19–21)

构造3D卷积:

```
kernel=3×3×3
padding=1
bias=False
stride可配置
```

保持空间/时间尺寸或按stride下采样。

`conv1x1x1` (23–25)

1×1×1投影卷积, 用于shortcut channel/stride对齐。

BasicBlock (27–47)

`expansion=1`。初始化建立：

```
conv3x3x3 → BatchNorm3d → ReLU
→ conv3x3x3 → BatchNorm3d
→ 加identity/downsample
→ ReLU
```

`forward`逐行含义：

1. `residual=x`保存shortcut;
2. 第一卷积、BN、ReLU;
3. 第二卷积、BN;
4. 若downsample存在，对residual投影;
5. 与主分支相加;
6. 最终ReLU并返回。

Bottleneck (49–72)

`expansion=4`，结构是1×1降/调channel、3×3处理、1×1扩展。当前ResNet3D-18使用BasicBlock，Bottleneck保留给更深模型。

ResNet.__init__ (75–119)

关键成员：

```
conv1: 7×7×7, input channels=3, output=64
bn1/relu/maxpool
layer1..layer4
avgpool: AdaptiveAvgPool3d((1,1,1))
fc: Linear(512*expansion, n_classes)
```

权重初始化：

- Conv3d使用Kaiming normal;
- BatchNorm scale=1、bias=0。

_downsample_basic_block (121–127)

用于shortcut type A：用平均池化降采样，再用零channel补齐。`out.data`切断该分支旧式autograd关系；当前配置应以构造参数中的shortcut type为准。

_make_layer (129–146)

决定是否需要downsample，然后：

- 第一个block处理stride/channel变化;
- 剩余blocks保持尺寸;
- 更新`self.in_planes`;
- 返回`nn.Sequential`。

forward_stem (148–150)

依次执行`conv1 → bn1 → relu → maxpool`，输出五维feature map。

forward_features (152–160)

执行stem和四个residual stages，adaptive average pool后`flatten(1)`：

```
[B, 512, 1, 1, 1] → [B, 512]
```

这是feature extraction使用的接口。

forward_head (162–165)

对[B, 512]先dropout再fc, 得到[B, n_classes]。

forward (167–169)

组合forward_features和forward_head。

generate_model (171–185)

按depth选择block与每stage block数。正式配置18对应BasicBlock [2, 2, 2, 2]。不支持的depth抛出 ValueError。

5. data.py (1–281)

safe_torch_load (18–22)

优先weights_only=True降低pickle对象加载风险；旧版PyTorch不支持该参数时捕获TypeError回退。统一map_location="cpu", 设备搬运在DataLoader之后完成。

uniform_frame_indices (25–28)

先检查原始帧数和目标帧数均大于0, 再用linspace(0, T_raw-1, T)生成均匀位置并转int64列表。当原视频少于16帧时索引可能重复, 这是均匀重采样而非报错。

RGBVideoTransform (31–69)

训练和评估构造两套torchvision.transforms.v2流水线。输入video_tchw为[T, 3, H, W], 包装成tv_tensors.Video保证同一个随机空间变换一致应用到所有帧。最终转普通Tensor并 contiguous()。

call (68–69) 使transform实例可以像函数一样调用；它不改变维度顺序, 只执行已构造的变换并 保证返回内存连续tensor。

RGBClipDataset.__init__ (73–101)

逐项动作:

1. 规范dataset root和manifest路径;
2. 保存camera、帧数;
3. 整体读取manifest;
4. 构造transform;
5. 可选逐样本检查<camera>_rgb;
6. 最多收集10个缺失示例后抛FileNotFoundError。

RGBClipDataset.__getitem__ (106–128)

1. 按index取manifest行;
2. 拼出RGB文件路径并安全加载;
3. 字典对象取frames, tensor对象直接使用;
4. 验证[T, 3, H, W];
5. 均匀取16帧;
6. 变换后从[T, 3, H, W]换为[3, T, H, W];
7. 返回视频、两个标签和样本metadata;
8. node_idx-1完成1-based→0-based转换。

HistoryExample (131–136)

冻结dataclass保存cache索引和manifest行, 不重复保存feature tensor。

`load_feature_cache` (139–146)

要求四个字段`features/tier3_logits/records/metadata`，并验证`record`数等于`feature`第一维。

`FeatureHistoryDataset.__init__` (150–212)

逐段：

- 校验`history_order ∈ {actual, graph_valid}`;
- `cache feature`强制`float32`;
- `graph-valid`模式必须提供`TaskGraphSpec`;
- 建立`sample_name → cache index`;
- `selection manifest`中任一样本不在`cache`则停止;
- 按`run_key`分组;
- 每组按`annotation index`排序;
- 当前位置之前的`rows`构成历史;
- `graph-valid`时用`sample`稳定`seed`重排;
- 将当前/历史`cache`索引封装为`HistoryExample`;
- 最终按`participant`、`run`、`annotation index`排序，保证评估输出稳定。

`feature_dim / __len__` (214–219)

`feature`维度取`features.shape[1]`；长度为构造出的当前样本数。

`__getitem__` (221–248)

- 有历史时用`long`索引`index_select`，结果`[1, F]`;
- 空历史创建同`dtype/device`的`[0, F]`;
- `position ID`为`1..1`，1代表`presented sequence`最后一个/最近历史;
- 历史`node`转0-based `long tensor`;
- 返回当前`feature`、历史、`targets`和可追溯`sample names`。

`collate_history_batch` (251–280)

先计算`B/F/L`，创建`padding`张量，再逐样本写入前`length`位置并把`mask`设`False`。最后`stack`当前`feature`、构造`target tensors`，同时保留字符串列表。该函数是所有M0–M6和Direct模型DataLoader的`collate_fn`。

6. graph.py (1–167)

`TaskGraphSpec.load` (29–89)

1. 读取两个JSON;
2. 建立1-based `node`字典;
3. 验证1..35完整存在;
4. 生成`node_to_tier3 [35]`;
5. 生成`node_to_stage [35]`;
6. 根据`relation JSON`显式列顺序建立`column lookup`;
7. 双循环构造`relation_ids [35, 35]`;
8. 规范化为`X`，未知关系立即报错;
9. 读取全部必须前序与立即前序;

10. 读取atomic sequences;

11. 返回不可变TaskGraphSpec。

关系矩阵赋值使用`current_node-1`, `previous_node-1`, 因此模型内部完全0-based。

`stable_sample_seed` (92–94)

把`base_seed:sample_name`做SHA-256, 取前8字节作为无符号整数。与Python内置`hash()`不同, 它不受 进程hash随机化影响, 所以同seed、同sample始终得到相同graph-valid排列。

`randomized_graph_valid_history` (97–167)

功能是对“已经观察到的历史rows”产生一个随机但合法的拓扑序:

- 0/1个历史直接复制返回;
- 历史出现重复node时回退实际顺序, 避免单个node ID无法区分多次实例;
- 只在已观察node之间建约束边;
- 使用`all_must_previous`和atomic sequence约束;
- `immediate_previous`虽然由`TaskGraphSpec.load`保存, 但当前排序函数没有单独读取它;
- Kahn式拓扑排序每一步从当前可用node中用本地`random.Random(seed)`选择;
- 若观察子图出现环或无法完成, 抛出`RuntimeError`, 不会静默改回实际顺序;
- 不接收当前目标node, 因此不会用当前标签决定历史排列。

7. `protocols.py` (1–144)

`find_fault_manifest` (12–22)

为指定participant定位`fault_run_test_manifest.jsonl`。尝试预定义候选位置; 找不到时抛出明确错误。

`load_global_fault_run_keys` (24–30)

对A/D/J/M读取fault manifests, 把每行转换为`(participant,run)`集合。集合去重保证一个fault run 无论含多少clip只记录一次。

`_validate_rows` (32–52)

检查协议行关键字段、node/Tier3/stage范围以及排序所需信息; 错误中携带source, 便于定位原manifest 还是生成协议。

`_sorted` (54–63)

返回新list, 按participant、run、annotation index和sample name稳定排序。

`_summary` (65–86)

统计samples、run数、node/Tier3支持度、缺失类别等, 供`protocol_report.json`使用。

`prepare_protocols` (88–144)

读取完整manifest并产生:

- held-out participant的test normal/fault/all;
- 其余participants的normal-only train;
- 其余participants的all-runs train;
- 各split摘要。

fault划分以run为单位, 而不是只移除fault run中的某几个clip, 防止同run泄漏到normal-only训练。

8. metrics.py (1–60)

confusion_matrix (9–14)

创建[K, K] int64零矩阵，逐样本把matrix[truth, pred] += 1。越界标签被忽略，正常数据验证应保证 不会发生。

metrics_from_confusion (17–42)

- support按行求和；
- predicted按列求和；
- tp取对角线；
- 安全除法计算recall/precision/F1，零分母输出0；
- present=support>0；
- accuracy对所有样本；
- macro-F1和balanced accuracy只平均present classes；
- 返回per-class数组、support和完整matrix。

classification_metrics (45–48)

把Python list转int64 NumPy数组，再组合前两个函数。

aggregate_node_probabilities (51–60)

创建前导维度不变、末维31的零tensor；把node_to_tier3reshape/expand到与node probabilities相同 shape；最后沿末维scatter_add_。输入输出保留device和dtype。

9. video_evaluation.py (1–203)

_stage_metrics (15–30)

按stage筛选真实/预测列表，对每个stage分别调用classification metrics。

_write_predictions (32–41)

确保目录存在，使用首行keys作为CSV header；空rows时不写数据行。

evaluate_tier3_video_model (43–109)

用于E2E-Tier3：

- @torch.no_grad()；
- 模型eval；
- 遍历RGB loader并把tensor移到device；
- 可选AMP；
- logits [B, 31]→softmax；
- 收集真实/预测、stage和逐样本metadata；
- 计算overall/per-stage Tier3；
- 写metrics、predictions和probability tensor；
- 不伪造node指标。

evaluate_node_video_model (111–203)

用于E2E-Node：

- RGB模型输出 [B, 35]；

- softmax得到node probabilities;
- 调用固定映射聚合 [B, 31];
- 同时收集node和Tier3;
- 写overall/per-stage、predictions以及两种概率。

来源文件: 03_models_and_engine_line_by_line.md

03 模型与训练Engine逐段解析

1. models.py

1.1 FeatureNodeClassifier (12–25)

初始化 (15–21)

```
self.feature_dim = feature_dim
self.num_nodes   = num_nodes
self.norm        = LayerNorm(feature_dim)
self.dropout     = Dropout(dropout)
self.fc          = Linear(feature_dim, num_nodes)
```

默认输入 [B, 512]，输出 [B, 35]。LayerNorm按每个样本最后一维归一化；Dropout只在train模式生效。

forward (23–25)

逻辑为：

```
features [B, 512]
→ norm [B, 512]
→ dropout [B, 512]
→ fc [B, 35]
```

没有softmax，因为cross_entropy需要raw logits；softmax只在评估或概率构造时调用。

1.2 freeze_module (27–31)

先eval()，再遍历所有parameters设置requires_grad=False。冻结同时包含：

- 不计算参数梯度；
- dropout关闭；
- 若模块包含BatchNorm，不更新running statistics。

1.3 SingleQueryHistoryModel (33–113)

__init__ (36–71)

输入参数：

参数	含义
baseline	已训练M0 FeatureNodeClassifier
feature_dim	512
d_model	256
num_heads	4
max_history	35
dropout	0.1
use_position	M1 False; M2/M3 True

构造顺序：

1. 保存并冻结baseline；
2. 当前与历史各用Linear(512, 256)+LayerNorm(256)投影；
3. 建立Embedding(36, 256)，ID 0给padding，1..35表示历史距离；
4. 建立可学习null token[1, 1, 256]；
5. 建立batch-first MultiheadAttention；

6. delta head读取`[current;context]` `[B, 512]`并输出`[B, 35]`;

7. `history_scale_logits`初始化为-2, 所以初始`sigmoid`≈0.119, 历史残差从较小幅度开始。

train (73–76)

调用父类切换整体模式后, 立即把baseline重新设为eval。这样即使外层`model.train()`, 冻结M0仍不启用 dropout。

forward (78–113)

输入shape:

```
current_feature      [B, 512]
history_features     [B, L, 512]
history_position_ids [B, L]
history_padding_mask [B, L]
```

逐段:

1. 当前投影`current` `[B, 256]`;
2. 历史投影`history` `[B, L, 256]`;
3. 若使用位置且 $L > 0$, clamp ID到0..35并加embedding;
4. null token扩展为`[B, 1, 256]`并拼到历史最前, 得到`[B, L+1, 256]`;
5. 拼一个全False null mask, 得到`[B, L+1]`;
6. query为`current.unsqueeze(1)` `[B, 1, 256]`, key/value为历史;
7. attention返回`context` `[B, 1, 256]`及weights;
8. `squeeze(1)`得到`[B, 256]`;
9. 拼接`current/context`为`[B, 512]`;
10. delta head输出`[B, 35]`;
11. `scale=sigmoid(history_scale_logits)`, 标量;
12. `no_grad`计算冻结baseline logits`[B, 35]`;
13. 最终`baseline + scale*delta`;
14. 返回logits和诊断字典。

null token使空历史batch的key长度仍为1; 这时模型可学习“无历史”的默认context。

1.4 DirectSingleQueryHistoryModel (116–197)

与SingleQuery相同的部分

- 当前/历史投影;
- 可选位置embedding;
- null token;
- MultiheadAttention;
- 空历史与padding处理。

Direct特有层 (151–160)

```
fusion = Linear(512+256, 512)
node_classifier = LayerNorm(512) → Dropout(0) → Linear(512, 35)
```

初始化先把fusion权重和bias清零, 然后把左侧 512×512 块复制为单位矩阵。classifier保持PyTorch默认 随机初始化。

forward (162–197)

1. 保留原始`current_feature` `[B, 512]`;
2. 投影当前query与历史;

3. 可选位置;
4. 添加null并做attention;
5. context从[B, 1, 256]变[B, 256];
6. 拼接[current_feature;context] [B, 768];
7. fusion得到[B, 512];
8. fused_feature送入FeatureNodeClassifier;
9. classifier内部LayerNorm、Dropout(0)和Linear得到[B, 35];
10. 返回logits和attention/context/fused_feature诊断值。

这里没有baseline、history scale或delta。

1.5 CandidateHistoryModel (200–371)

初始化 (203–254)

关键检查:

- `d_model % num_heads == 0;`
- `graph_source`只能是`none/oracle/predicted`。

关键成员:

<code>baseline</code>	frozen M0
<code>relation_ids</code>	buffer [35, 35]
<code>current_projection</code>	512→256
<code>history_projection</code>	512→256
<code>position_embedding</code>	36×256
<code>candidate_embedding</code>	35×256
<code>query/key/value projections</code>	256→256
<code>relation_bias</code>	[heads, 5]
<code>immediate_not_last_bias</code>	[heads]
<code>null_history</code>	[1, 1, 256]
<code>delta_head</code>	对每个candidate输出标量

`register_buffer`让relation IDs随模型移动设备并写入state dict, 但不参与梯度。

`train` (256–259)

与SingleQuery相同: 整体切train后强制baseline为eval。

`_history_node_probabilities` (261–280)

按graph source分支:

- `none`: 返回全零[B, L, 35];
- `oracle`: 对`history_node_classes`做one-hot; 先clamp避免padding=-1报错, 再用mask清零padding;
- `predicted`: 展平历史为[B*L, 512], 冻结baseline输出[B*L, 35], softmax后reshape [B, L, 35], padding清零;

- `L=0`时维持合法空shape。

M6训练时读取的是M0对历史clip的当前预测, 不是历史真实node。

`_graph_bias` (282–310)

输入:

<code>history_probabilities</code>	[B, L, 35]
<code>position_ids</code>	[B, L]
<code>padding_mask</code>	[B, L]

1. `none`或空历史返回[B, A, 35, L]零tensor;
2. `self.relation_bias[:, self.relation_ids]`得到pair_bias [A, 35, 35];

3. einsum将历史node概率对最后一个历史node维加权求和:

```
graph_bias[b,h,v,l] =  $\sum_u p\_history[b,l,u] * bias[h, relation(v,u)]$ 
```

4. 单独计算历史token成为候选node立即前序的概率;

5. 若该token位置ID不等于1, 则加immediate_not_last_bias惩罚/奖励;

6. 返回每head、每候选node、每历史位置的bias。

forward (312–371)

1. 取B/L;

2. 当前、历史投影并加位置;

3. candidate embedding扩展为[B, 35, 256];

4. 当前投影与candidate相加, 再投影为query;

5. null与历史拼接, 投影key/value;

6. reshape和transpose拆成4个head:

```
Q [B, 4, 35, 64]
```

```
K, V [B, 4, L+1, 64]
```

7. 点积/根号64得到scores[B, 4, 35, L+1];

8. 构造history node probabilities和graph bias;

9. 为null位置补零bias并与历史bias拼接;

10. 拼接padding mask并把不可读位置设为dtype最小有限值;

11. softmax + dropout得到attention;

12. attention乘values得到context;

13. 合并heads为[B, 35, 256];

14. 拼current expanded/context/candidate, 逐candidate输出delta[B, 35];

15. 冻结baseline输出;

16. baseline + scale×delta;

17. 返回logits和完整诊断字典。

1.6 build_context_model (374–404)

工厂映射:

```
m1 → SingleQuery(use_position=False)
m2/m3 → SingleQuery(use_position=True)
m4 → Candidate(graph_source=None)
m5 → Candidate(graph_source=oracle)
m6 → Candidate(graph_source=predicted)
```

M2与M3模型结构相同, 区别来自Dataset的history order。

1.7 build_direct_context_model (407–429)

```
m1_direct → DirectSingleQuery(use_position=False)
m2_direct/m3_direct → DirectSingleQuery(use_position=True)
```

M2/M3 Direct结构相同, 仍由Dataset决定actual或graph-valid。

2. engine.py

2.1 move_batch_to_device (17–21)

字典推导逐key处理:

- Tensor调用.to(device, non_blocking=True);

- 字符串列表等metadata原样保留。

返回新dict，不原地修改raw batch。

2.2 forward_node_model (24–34)

- 若模型是M0 `FeatureNodeClassifier`，只传`current_feature`并返回空诊断字典；
- 其他history/direct模型按命名参数传五个batch tensor；
- 统一返回(`logits`, `diagnostics`)，让训练循环不需要为模型族写多个分支。

2.3 compute_loss (37–58)

输入：

```
logits      [B, 35]
node_target [B]
tier3_target [B]
mapping     [35]
```

逐行：

1. node交叉熵；
2. 默认总loss=node loss；
3. 创建同device/dtype的0标量action loss；
4. 权重大于0时softmax node；
5. 聚合到[B, 31]；
6. `gather`每个样本真实Tier3概率，结果[B, 1]→[B]；
7. `clamp_min(1e-12)`避免log(0)；
8. 负对数均值；
9. 加权加入总loss；
10. 返回可反传loss和已detach的日志标量。

2.4 train_feature_model (61–116)

初始化AMP GradScaler、history list并把mapping移到device。每个epoch：

1. `model.train()`；history模型内部会保持baseline eval；
2. 初始化计时、loss、correct、total；
3. 遍历loader并搬device；
4. `zero_grad(set_to_none=True)`减少内存写；
5. AMP上下文内forward和loss；
6. scaler缩放后backward；
7. unscale；
8. 对所有可训练参数做global norm裁剪，阈值1.0；
9. optimizer step与scaler update；
10. loss按batch size加权累积；
11. argmax node计数；
12. epoch结束记录平均loss、accuracy和秒数；
13. 打印并append；
14. 所有epoch后返回训练日志list。

没有validation、scheduler、early stopping或best checkpoint逻辑。

2.5 evaluate_feature_model (120-216)

装饰器`@torch.no_grad()`关闭梯度。流程：

1. eval模式；
2. mapping移到device；
3. 初始化真实/预测/stage/row/probability容器；
4. 对每个batch统一forward；
5. node softmax[B, 35]；
6. 聚合Tier3[B, 31]；
7. 取得两个argmax；
8. node probabilities移CPU收集；
9. 逐样本构造可追溯prediction row；
10. 计算overall node/Tier3；
11. 对stage 1/2/3筛选索引并分别计算；
12. 写*_metrics.json；
13. 写*_predictions.csv；
14. 写*_probabilities.pt, 含[S, 35]和rows；
15. 返回metrics。

置信度是预测类别的最大概率，不是校准后的概率保证。

来源文件: 04_tools_reference.md

04 tools命令行入口逐文件参考

所有入口都会把包根加入`sys.path`，因此可从包根直接运行`python tools/<name>.py`。BAT/Slurm负责 提供长路径和统一参数；直接手工调用时必须自行保证scope、cache和checkpoint匹配。

1. 总表

文件	主要职责	是否训练	主要写入
<code>validate_setup.py</code>	环境、manifest、graph、checkpoint检查	否	控制台
<code>prepare_protocols.py</code>	生成LOSO协议	否	protocols JSONL/JSON
<code>guard_output_dir.py</code>	shell前置防覆盖检查	否	可创建空目录
<code>train_backbone.py</code>	训练31类Tier3 ResNet3D	是	backbone scope目录
<code>extract_features.py</code>	提取512维feature cache	否	.pt cache
<code>train_history_model.py</code>	训练M0–M6	是	history model目录
<code>train_direct_history_model.py</code>	训练M1–M3 Direct	是	direct_head_fusion目录
<code>evaluate_e2e_tier3.py</code>	复测已有Tier3 checkpoint	否	E2E Tier3评估
<code>train_e2e_node.py</code>	训练两种35-node E2E	是	E2E node目录
<code>smoke_test_models.py</code>	合成tensor测试M1–M6	短反传	临时内存
<code>smoke_test_direct_models.py</code>	测试Direct模型与初始化	短反传	临时内存
<code>summarize_results.py</code>	单fold旧M0–M6汇总	否	CSV
<code>summarize_cross_person.py</code>	早期跨人汇总	否	CSV
<code>summarize_all_models.py</code>	十模型严格统一汇总	否	多个CSV
<code>summarize_direct_head_fusion.py</code>	Direct专用严格汇总	否	3 CSV + completed

2. `validate_setup.py` (52行)

只有`main()`。

流程：

1. 解析dataset、task graph、relation matrix、test participant、camera和可选checkpoint；
2. 检查路径存在；
3. 加载TaskGraphSpec，验证35 nodes和relation；
4. 定位输入manifest并检查相机字段；
5. 若给checkpoint，尝试兼容加载；
6. 输出检查摘要。

它不生成协议、不训练，也不修改已有输出。适合换电脑/HPC后第一步执行。

3. `prepare_protocols.py` (53行)

只有`main()`。

参数：

```
--dataset-root
--output-root
--test-participant {A, D, J, M}
--camera-id
```

流程:

1. parse;
2. 调用`prepare_protocols(...)`;
3. 写normal-only/all-runs训练manifest与三个test manifests;
4. 写`protocol_report.json`;
5. 打印每个split规模。

协议生成是确定性的，同一dataset与participant应得到相同内容。

4. `guard_output_dir.py` (24行)

只有`main()`。接收`--output-dir`，调用非覆盖安全逻辑。它用于BAT/Slurm在启动昂贵任务前提前失败，避免训练很久后才发现目标目录冲突。

5. `train_backbone.py` (163行)

`train_epoch` (30–53)

- `model.train()`;
- 遍历RGB batch;
- video/target搬device;
- AMP前向;
- Tier3 cross entropy;
- GradScaler backward/step/update;
- 累积按样本加权loss和accuracy;
- 返回epoch指标。

输入video[B, 3, 16, 224, 224], logits[B, 31]。

`evaluate` (55–89)

在eval/no-grad下测试Tier3模型，收集概率和metadata并写metrics/predictions/probabilities。该内部函数 用于训练结束后的测试，不进行checkpoint选择。

`main` (91–163)

关键参数:

```
--dataset-root --protocol-root --train-scope
--output-dir --camera-id
--epochs --batch-size --num-workers
--learning-rate --weight-decay --seed --device
--amp/--no-amp --overwrite
```

执行:

1. seed和device;
2. `ensure_new_output_dir`;
3. 根据scope选择train manifest;
4. 构造RGB datasets/loaders;
5. `generate_model(18, n_classes=31)`;
6. AdamW、GradScaler和`MultiStepLR(milestones=[50, 75], gamma=0.1)`;
7. 训练固定epochs，每个epoch结束推进scheduler;

8. 保存last.pth和日志;
9. 测三个split;
10. 写completed.json。

6. extract_features.py (108行)

只有main()。

参数包括dataset、manifest、Tier3 checkpoint、output、camera、batch size、workers、AMP、seed、 debug sample上限、completion marker和overwrite。

执行:

1. 保护输出文件/目录;
2. 构造非训练RGB dataset;
3. 生成31类ResNet3D并兼容加载checkpoint;
4. eval/no-grad;
5. 对每个batch调用forward_features得到[B, 512];
6. forward_head得到[B, 31];
7. 收集records;
8. CPU拼接后保存:

```
features [S, 512]
tier3_logits [S, 31]
records length S
metadata
```

9. 可选写completion marker。

它不会更新backbone权重。

7. train_history_model.py (190行)

build_loader (30-40)

构造DataLoader:

- dataset为FeatureHistoryDataset;
- collate_fn=collate_history_batch;
- shuffle由训练/测试决定;
- pin memory只在CUDA时启用;
- persistent workers依赖num_workers>0。

main (42-190)

主要参数:

```
--model {m0..m6}
--train-scope
--protocol-root
--train-cache --test-cache
--task-graph --relation-matrix
--baseline-checkpoint
--output-root
--epochs --batch-size --num-workers
--learning-rate --weight-decay
--d-model --num-heads --max-history --dropout
--action-loss-weight --seed --device --amp --overwrite
```

分支：

- M0：建立FeatureNodeClassifier，不需要baseline checkpoint；
- M1–M6：先建立同结构M0并严格加载指定baseline，然后由工厂构造history模型；
- M3使用`history_order=graph_valid`；
- 其余使用actual；
- M5/M6结构由model factory选择oracle/predicted。

训练集使用scope对应manifest和train cache。测试cache可覆盖held-out participant全部样本，但三个测试 Dataset分别用三个 selection manifest过滤。

输出：

```
last.pth
train_log.json
learned_parameters.json
experiment_config.json
test_results/*
completed.json
```

`learned_parameters.json`记录history scale、relation bias等可解释参数；M0相应字段较少。

8. `train_direct_history_model.py` (206行)

`build_loader` (35–45)

与history loader相同。

`main` (47–206)

模型限制为`m1_direct/m2_direct/m3_direct`。没有`--baseline-checkpoint`，这是与原history入口的关键 接口差异。

执行：

1. 解析scope和cache；
2. seed/device/output防覆盖；
3. 加载TaskGraphSpec；
4. M3 Direct选择graph-valid，其余actual；
5. 构造train和三个test FeatureHistoryDataset；
6. 从train dataset取得feature_dim；
7. `build_direct_context_model`；
8. AdamW训练固定epochs；
9. 测normal/fault/all；
10. 从feature cache metadata记录Tier3 representation来源；
11. 写checkpoint、配置、learned parameters和completed。

checkpoint不包含M0参数。`learned_parameters.json`应重点查看fusion和attention相关摘要。

9. `evaluate_e2e_tier3.py` (105行)

只有`main()`。加载已有31类Tier3 `last.pth`并重新对三个split评估。它：

- 不训练；
- 不复制或修改原checkpoint；
- 使用RGBClipDataset；

- 调用`evaluate_tier3_video_model`;
- node指标为空/不存在;
- 专属新目录受防覆盖和completed保护。

10. `train_e2e_node.py` (205行)

`make_loader` (31–40)

普通RGB DataLoader，无history collate。

`train_epoch` (42–64)

与backbone训练类似，但target改为35-node，logits`[B, 35]`。

`main` (66–205)

模式由参数决定：

- scratch：随机初始化35-node ResNet3D;
- from-tier3：生成35-node模型，再兼容加载31类checkpoint，最终fc因shape不同跳过。

固定epoch训练全网络；优化器是AdamW，scheduler为`MultiStepLR(milestones=[50, 75], gamma=0.1)`，每batch做梯度范数裁剪1.0。最后用`evaluate_node_video_model`同时报告node和Tier3。

11. Smoke tests

`smoke_test_models.py`

构造合成：

```
current [B, 512]
history [B, L, 512]
positions/nodes/mask
```

逐个M1–M6检查：

- forward shape `[B, 35]`;
- loss有限;
- backward可运行;
- baseline无梯度;
- 空历史可运行;
- graph bias诊断shape正确。

`smoke_test_direct_models.py`

逐个Direct模型检查：

- forward/backward;
- 空历史;
- fusion初始时history权重块为0;
- current权重块为单位矩阵;
- 输出35 logits;
- 没有冻结M0依赖。

Smoke test只验证代码路径和shape，不代表模型性能。

12. summarize_results.py (54行)

只有main()。扫描一个fold/seed下M0-M6的三个metrics JSON，展平核心指标后写CSV。属于早期单fold 汇总，严格四折最终报告优先使用summarize_all_models.py。

13. summarize_cross_person.py (182行)

小函数

- mean: 算术平均;
- sample_std: 分母n-1, 少于2项返回0;
- write_csv: 完整写出, 不追加。

collect_rows

扫描指定participants的早期cross-person目录，读取M0-M6 metrics并附上participant/model/split。

build_deltas

按participant与split配对，计算各模型减M0。

build_aggregate

先在participant内聚合重复seed，再跨participant均值/样本标准差。

main

解析root/output/participants，依次写metrics、deltas和aggregate。

14. summarize_all_models.py (583行)

这是正式十模型和严格scope比较的主汇总器。

write_csv (46-53)

确保父目录，按rows第一行字段完整写CSV；空rows创建空文件/按实现处理。

metric_value (55-60)

从metrics嵌套字典安全取node/tier3指标。直接Tier3模型的node返回None。

parse_location (62-99)

根据metrics文件相对outputs_root的路径解析：

```
participant
camera
seed
model
train_scope
representation_scope
split
```

这是防止normal-only表示与all-runs训练条件被误合并的关键。

collect_rows (101-151)

遍历metrics JSON，解析位置，筛选participants/seeds/scopes，生成overall长表。

collect_stage_rows (153-209)

把每个metrics的Stage 1/2/3展开为三行，并保留present class count。

require_complete_grid (211-251)

在写汇总前检查请求网格。缺少任一participant、seed、scope、10 models或3 splits就列出缺项并抛错。

build_pairwise_deltas (253–303)

在相同participant/seed/scope/split内按可比较reference配对。不会跨seed或跨representation scope。

mean_or_none / std_or_none (305–311)

过滤None后聚合；直接Tier3的node指标保持空值。

aggregate_across_people (313–359)

第一层按participant平均seeds，第二层对participants等权平均并给样本标准差。

aggregate_stages_across_people (361–372)

相同规则用于per-stage。

aggregate_deltas (374–428)

对严格模型差值先participant内平均seed，再跨participant。

build_training_scope_deltas (430–463)

配对键：

```
participant + seed + model + split
```

计算：

```
all_runs - normal_only
```

并要求representation scope与train scope匹配。

aggregate_training_scope_deltas (465–502)

同样采用participant内seed平均、再participant等权。

main (504–583)

控制筛选、完整网格、所有CSV写入与completed marker。严格四折报告的主要源文件由这里产生。

15. summarize_direct_head_fusion.py (287行)

mean / sample_std / write_csv

与cross-person工具相同的基础统计与CSV输出。

read_metric (41–53)

读取一个metrics JSON并提取6个核心指标：

```
node accuracy/macro-F1/balanced
tier3 accuracy/macro-F1/balanced
```

legacy_model_root (55–60)

根据scope返回原M0–M3的正确目录：

- normal-only → history_models/retrained_normal_only/normal_only
- all-runs → history_models/retrained_all_runs/all_runs

collect_rows (62–141)

对4人×3seed×2scope×3direct×3split：

1. 读取Direct metrics生成216行；
2. 读取同scope M0和对应原M1/M2/M3；
3. 生成每个Direct对两个references的432行严格差值；

4. complete-grid开启时任何缺失立即停止。

`build_aggregate` (143–224)

分组键：

```
scope + direct model + reference + split
```

先participant内平均3 seeds，再跨4人均值与样本标准差，同时保留Direct绝对指标与delta。

`main` (226–287)

默认participants A/D/J/M、seeds 1/2/42、两个scope；写：

```
direct_head_metrics.csv  
direct_head_paired_deltas.csv  
direct_head_aggregate.csv  
completed.json
```

来源文件: 05_bat_slurm_reference.md

05 Windows BAT、Shell与Slurm逐脚本参考

1. 如何阅读这些脚本

脚本层不实现模型数学；它负责：

- 设置路径、participant、seed、scope和超参数；
- 检查依赖是否完成；
- 把长参数传给tools/*.py；
- 建立顺序或Slurm afterok依赖；
- 遇到非零退出码立即停止；
- 把不同实验写进互相隔离的目录。

2. BAT常见语句

语句	含义
@echo off	不回显每条命令
setlocal	环境变量修改限制在当前脚本
call config_windows.bat	导入公共路径/默认值
if errorlevel 1 exit /b 1	上一步失败则停止
if exist completed.json ...	完成后安全跳过
call other.bat	在同一cmd进程调用子流程并返回
"%PYTHON_BIN%" ...	使用带空格安全引用的指定Python
~	Windows命令续行

3. Windows配置与基础1–12

BAT	作用
config_windows.bat	定义package root、dataset、Python、assets、outputs和默认超参数
00_validate_setup.bat	调用validate_setup.py检查当前fold
01_prepare_protocols.bat	为当前TEST_PARTICIPANT生成协议
02_train_backbone_normal_only.bat	从scratch训练normal-only Tier3 backbone
03_extract_features_retrained_last.bat	用normal-only last.pth提取train/test cache
04_train_main_m0_m6.bat	先M0，再顺序训练M1–M6
05_train_aux_all_runs_m0_m6.bat	旧辅助条件：normal-only representation + all-runs history训练
06_summarize_results.bat	单fold/seed早期M0–M6汇总
07_summarize_cross_person.bat	早期A/D/M跨人汇总
08_evaluate_e2e_tier3_existing.bat	复测normal-only Tier3 checkpoint
09_train_e2e_node_scratch.bat	normal-only 35-node E2E scratch
10_train_e2e_node_from_tier3.bat	normal-only Tier3迁移35-node
11_summarize_all_models_fold.bat	单fold/seed十模型统一汇总
12_summarize_all_models_cross_person.bat	早期多fold十模型汇总

4. 完整all-runs 13–23

BAT	作用
13_prepare_protocols_all_runs_safe.bat	协议完整则复用、部分存在则停止
14_train_backbone_all_runs.bat	独立all-runs Tier3 backbone
15_extract_features_all_runs.bat	用all-runs backbone提取独立cache
16_train_all_runs_m0_m6.bat	用all-runs cache训练M0–M6
17_evaluate_e2e_tier3_all_runs.bat	复测all-runs Tier3
18_train_e2e_node_scratch_all_runs.bat	all-runs E2E node scratch
19_train_e2e_node_from_tier3_all_runs.bat	all-runs Tier3迁移node
20_summarize_all_runs_fold.bat	单fold all-runs+模型
21_summarize_training_scope_comparison_fold.bat	单fold all-runs normal-only
22_summarize_all_runs_cross_person.bat	all-runs跨人
23_summarize_training_scope_comparison_cross_person.bat	跨人scope差值

5. 严格四折多seed 24–30

BAT	作用
24_train_backbone_normal_only_safe.bat	带completed保护的normal-only backbone
25_extract_features_normal_only_safe.bat	带cache/marker检查的feature extraction
26_train_normal_only_m0_m6_safe.bat	安全训练normal-only M0–M6
27_summarize_normal_only_fold.bat	单fold/seed normal-only+模型
28_summarize_normal_only_ADJM_3seeds.bat	严格4人×3seed normal-only
29_summarize_all_runs_ADJM_3seeds.bat	严格4人×3seed all-runs
30_summarize_training_scope_comparison_ADJM_3seeds.bat	严格360条scope配对

6. Direct Head Fusion 31–33

BAT	作用
31_train_direct_head_fusion_normal_only.bat	当前participant/seed运行3个normal Direct模型
32_train_direct_head_fusion_all_runs.bat	当前participant/seed运行3个all-runs Direct模型
33_summarize_direct_head_fusion_ADJM_3seeds.bat	检查216行网格并生成Direct汇总

Direct训练脚本复用已有feature cache，不调用backbone/M0训练。A/D/M seed1旧normal cache缺少后来新增的 completed marker时，脚本通过显式验证两个.pt文件允许安全复用。

7. Windows组合入口

BAT	完整调用链
run_one_fold.bat	validate→protocol→normal backbone→features→M0–M6→可选aux→summary
run_all_ADM.bat	顺序运行A/D/M run_one_fold
run_additional_e2e_one_fold.bat	只运行3个E2E和新汇总，不重跑M0–M6
run_additional_e2e_ADM.bat	A/D/M增量E2E
run_all_runs_one_fold.bat	单fold完整all-runs pipeline
run_all_runs_ADM.bat	A/D/M完整all-runs
run_normal_only_complete_one_fold.bat	单fold/seed严格normal-only完整链
run_normal_only_multiseed_ADM.bat	A/D/M补seed 2/42
run_strict_J_one_seed.bat	J一个seed的两个scope严格scratch
run_strict_J_three_seeds.bat	J的1/2/42
run_fourfold_ADJM_summaries.bat	只重建三个严格四折汇总
run_recommended_strict_experiments.bat	补齐严格网格并汇总
run_direct_head_fusion_one_fold.bat	当前participant/seed两个scope的6个Direct
run_direct_head_fusion_ADJM.bat	4人×3seed×2scope×3模型并最终汇总

8. Slurm作业脚本 01–12

Slurm	GPU/CPU性质	对应任务
01_prepare_protocols.slurm	CPU	协议
02_train_backbone_normal_only.slurm	GPU	normal Tier3 backbone
03_extract_features.slurm	GPU	normal features
04_train_m0.slurm	GPU/轻量	normal M0
05_train_context_models.slurm	array	M1–M6
06_summarize_results.slurm	CPU	fold汇总
07_summarize_cross_person.slurm	CPU	跨人汇总
08_evaluate_e2e_tier3_existing.slurm	GPU	Tier3复测
09_train_e2e_node_scratch.slurm	GPU	node scratch
10_train_e2e_node_from_tier3.slurm	GPU	node迁移
11_summarize_all_models_fold.slurm	CPU	十模型fold
12_summarize_all_models_cross_person.slurm	CPU	十模型跨人

9. Slurm完整all-runs 13–24

Slurm	任务
13_prepare_protocols_all_runs_safe.slurm	安全协议
14_train_backbone_all_runs.slurm	all-runs backbone
15_extract_features_all_runs.slurm	all-runs features
16_train_all_runs_m0.slurm	all-runs M0
17_train_all_runs_context_models.slurm	all-runs M1–M6 array
18_evaluate_e2e_tier3_all_runs.slurm	all Tier3复测
19_train_e2e_node_scratch_all_runs.slurm	all node scratch
20_train_e2e_node_from_tier3_all_runs.slurm	all node迁移
21_summarize_all_runs_fold.slurm	fold all summary
22_summarize_training_scope_comparison_fold.slurm	fold scope delta
23_summarize_all_runs_cross_person.slurm	跨人all
24_summarize_training_scope_comparison_cross_person.slurm	跨人scope delta

10. Slurm严格补齐与Direct 25–35

Slurm	任务
25_train_backbone_normal_only_safe.slurm	strict normal backbone
26_extract_features_normal_only_safe.slurm	strict normal features
27_train_normal_only_m0.slurm	strict normal M0
28_train_normal_only_context_models.slurm	strict normal M1–M6
29_summarize_normal_only_fold.slurm	strict normal fold
30_summarize_normal_only_ADJM_3seeds.slurm	四折normal
31_summarize_all_runs_ADJM_3seeds.slurm	四折all
32_summarize_training_scope_comparison_ADJM_3seeds.slurm	四折scope delta
33_train_direct_head_fusion_normal_only.slurm	array 1–3 normal Direct
34_train_direct_head_fusion_all_runs.slurm	array 1–3 all Direct
35_summarize_direct_head_fusion_ADJM_3seeds.slurm	Direct完整汇总

Direct array映射：

```
SLURM_ARRAY_TASK_ID=1 → m1_direct
SLURM_ARRAY_TASK_ID=2 → m2_direct
SLURM_ARRAY_TASK_ID=3 → m3_direct
```

11. HPC配置和提交脚本

config_hpc.sh

集中定义dataset、package/output root、module/conda环境、camera、epoch、batch size和资源参数。被作业与 submit脚本source。换HPC时优先修改此文件。

单fold提交

Shell	依赖图
submit_one_fold.sh	prepare→backbone→features→M0→M1-M6→summary
submit_aux_one_fold.sh	已有 features→aux M0/M1-M6→summary
submit_additional_e2e_one_fold.sh	3个E2E并行→fold summary
submit_all_runs_one_fold.sh	protocol后backbone/history/E2E分支→summary→scope compare
submit_normal_only_one_fold.sh	strict normal完整链
submit_strict_J_one_seed.sh	J同seed normal与all完整链
submit_direct_head_fusion_one_fold.sh	normal/all各一个3-task array， 可选both

多fold提交

Shell	作用
submit_ADM.sh	A/D/M主链并最终跨人汇总
submit_additional_e2e_ADM.sh	A/D/M增量E2E
submit_all_runs_ADM.sh	A/D/M all-runs
submit_normal_only_multiseed_ADM.sh	A/D/M normal seed2/42
submit_strict_J_three_seeds.sh	J三个seed
submit_recommended_strict_experiments.sh	推荐严格补齐总入口
submit_direct_head_fusion_ADJM.sh	提交24个arrays并以afterok等待全部完成后汇总

12. Slurm依赖语义

提交脚本保存`sbatch --parsable`返回的job ID，并用：

`--dependency=afterok:<jobid>`

只有上游成功退出才启动下游。多个独立分支用冒号连接job IDs。Direct总入口的汇总依赖24个array jobs 全部成功；任何一个participant/seed/scope的三个模型任务失败，最终summary不会运行。

13. 不应混用的入口

- 已完成严格结果后，不要为新增E2E调用旧`run_one_fold.bat`；
- Direct实验不要调用M0或backbone脚本；
- 完整all-runs不要误用旧辅助`05_train_aux_all_runs_m0_m6.bat`；
- 正式四折汇总不要用只默认A/D/M的早期cross-person入口；
- 只想重新汇总时直接调用28-30或33，不要重跑训练链。

来源文件：06_direct_head_fusion_results.md

06 Direct Head Fusion实现与新增结果

1. 名称

代码、配置、输出目录和完成标记统一使用Direct Head Fusion或direct_head_fusion。本项目中没有 名为direction fusion的模型；若口头使用“direction fusion head”，本文按Direct Head Fusion理解。

2. 研究问题

原M1–M3先训练M0的35-node classifier，再冻结M0并学习logit residual：

```
final_logits = frozen_M0_logits + sigmoid(scale) * history_delta
```

这意味着35-node决策经历两个50-epoch阶段：

```
先训练M0 node head
→ 冻结M0
→ 再训练history delta
```

Direct版本检验：

```
冻结Tier3视觉feature
→ 当前feature与history context融合
→ 随机初始化35-node head直接输出最终logits
```

它不加载M0，不受M0 logit空间约束，fusion与node head在同一个50-epoch阶段联合训练。

3. 三个模型

模型	历史顺序	位置embedding	分类机制
m1_direct	actual	无	融合后直接35-node head
m2_direct	actual	有	融合后直接35-node head
m3_direct	graph-valid	有	融合后直接35-node head

4. 初始化

fusion为Linear(512+256, 512)。初始化：

```
W[:, 0:512] = I512
W[:, 512:768] = 0
b = 0
```

所以初始时：

```
fusion([current_feature; context]) = current_feature
```

紧随其后的LayerNorm和随机node head仍参与训练。这个初始化的意义是让history context从零贡献起步，避免随机context在训练初期立即破坏Tier3视觉feature。

5. 结果完整性

实际读取：

```
outputs/direct_head_fusion_summary_ADJM_3seeds/completed.json
outputs/direct_head_fusion_summary_ADJM_3seeds/direct_head_metrics.csv
outputs/direct_head_fusion_summary_ADJM_3seeds/direct_head_paired_deltas.csv
outputs/direct_head_fusion_summary_ADJM_3seeds/direct_head_aggregate.csv
```

完成标记记录：

```
4 participants × 3 seeds × 2 scopes × 3 models × 3 splits = 216 metric rows
216 rows × 2 references = 432 paired-delta rows
36 aggregate rows
```

聚合方式：

1. 每位participant内部先平均seed 1/2/42;
2. 再对A/D/J/M四人等权平均;
3. $\pm$ 是四个participant均值之间的样本标准差。

6. test_all主要结果

6.1 normal-only

模型	Node Acc	Node Macro-F1	Tier3 Acc	相对M0 Node	相对原同名模型 Node
m1_direct	80.52 $\pm$ 4.93	80.86	83.16 $\pm$ 4.80	+13.76	+1.93
m2_direct	88.64 $\pm$ 3.50	85.64	89.18 $\pm$ 3.56	+21.88	+9.34
m3_direct	88.72 $\pm$ 3.97	85.51	89.29 $\pm$ 3.96	+21.96	+9.42

相对原模型的Tier3 Accuracy:

m1_direct - M1 = +2.14个百分点
m2_direct - M2 = +8.10个百分点
m3_direct - M3 = +8.31个百分点

6.2 all-runs

模型	Node Acc	Node Macro-F1	Tier3 Acc	相对M0 Node	相对原同名模型 Node
m1_direct	79.99 $\pm$ 6.52	80.69	84.97 $\pm$ 4.96	+10.19	+0.32
m2_direct	90.57 $\pm$ 2.66	87.81	90.64 $\pm$ 2.64	+20.76	+6.41
m3_direct	90.05 $\pm$ 3.31	87.60	90.27 $\pm$ 3.10	+20.25	+5.31

相对原模型的Tier3 Accuracy:

m1_direct - M1 = +0.12个百分点
m2_direct - M2 = +5.65个百分点
m3_direct - M3 = +4.64个百分点

7. 新结果说明什么

7.1 Direct head对带位置模型非常有效

m2_direct和m3_direct的提升远大于m1_direct。这表明性能来源不能简单描述成“重新训练一个 分类头”，因为没有位置embedding的M1 Direct只略高于原M1，而M2/M3 Direct提高5到9个百分点Node。

更合理的解释是：

- 位置结构让attention context携带强流程定位信息；
- 直接在融合后的512维空间训练node head，比把历史限制为冻结M0 logits上的residual更容易使用

这类信息；

- 原delta设计可能受冻结M0决策边界和history scale约束。

7.2 all-runs下M2 Direct略高于M3 Direct

在test_all:

M2 Direct Node Accuracy - M3 Direct = +0.52个百分点
M2 Direct Tier3 Accuracy - M3 Direct = +0.37个百分点

差距很小，且当前表是四participant平均，不应据此宣称actual order必然优于graph-valid order。应进一步查看participant/seed配对、run级结果和置信区间。

7.3 normal-only下两者几乎相同

M3 Direct - M2 Direct:
Node Accuracy +0.08个百分点
Tier3 Accuracy +0.11个百分点

这与原M2/M3在normal-only中接近的现象一致：正常流程顺序高度规律，actual与graph-valid历史提供的 结构非常相似。

7.4 scope效应不是所有模型都一致

直接从216行metric重新按participant内seed平均后计算all-runs - normal-only:

模型	test_all Δ Node	四位participant各自 Δ
m1_direct	-0.53	-5.03, -5.48, +4.08, +4.33
m2_direct	+1.93	-0.39, +0.72, +1.92, +5.44
m3_direct	+1.33	-1.08, -0.72, +1.38, +5.74

因此Direct模型的all-runs优势小于原M3的+5.44，且M2/M3 Direct并非四位participant全部为正。这部分属于描述性重新计算，当前专用summary没有正式输出scope-delta CSV；若用于论文，应由独立、可复现的统计脚本生成并加入置信区间。

8. 当前可支持的结论

可以支持：

- Direct Head Fusion完整网格已完成；
- M2/M3 Direct明显优于M0和原M2/M3；
- 直接feature fusion比冻结M0 logit delta更适合当前带位置的history表示；
- all-runs总体最佳Direct配置是M2 Direct，normal-only的M2/M3 Direct几乎并列。

暂不能支持：

- Direct模型在新participant总体上具有统计显著优势；
- actual order确定优于graph-valid order；
- all-runs对每位participant都改善Direct模型；
- Direct Head Fusion已经能够检测流程异常；
- 216个模型结果或12个participant-seed可当作独立受试者。

9. 建议后续分析

1. 对m2_direct - m2、m3_direct - m3做participant-run层级paired bootstrap。
2. 分stage检查提升是否仍集中在Stage 2重复动作node。
3. 聚合prediction，重新计算四组重复node双向混淆。
4. 对比Direct与delta版本的attention entropy、history scale和fusion权重范数。
5. 检查fusion矩阵的history部分 $W[:, 512:768]$ 是否学到非零、稳定结构。
6. 分析M2 Direct与M3 Direct差异是否由fault run中的重复/非法顺序驱动。

来源文件: 07_outputs_reproducibility_and_safety.md

07 输出、复现与安全边界

1. 单participant/seed根目录

outputs/<P>_as_test/cam_001484412812/seed_<S>/

$P \in \{A, D, J, M\}$, $S \in \{1, 2, 42\}$ 。

2. representation与模型目录

```
backbone/
├── normal_only/
└── all_runs/

features/
├── retrained_normal_only/
└── retrained_all_runs/

history_models/
├── retrained_normal_only/normal_only/m0..m6/
├── retrained_all_runs/all_runs/m0..m6/
└── direct_head_fusion/
    ├── normal_only/m1_direct..m3_direct/
    └── all_runs/m1_direct..m3_direct/

e2e_baselines/
├── normal_only/
└── all_runs/
```

目录名中的两个scope含义不同：

- `retrained_*`描述feature/backbone representation来源；
- 内层`normal_only/all_runs`描述history模型训练manifest。

正式matched-scope比较要求二者相同。

3. 模型目录文件

典型history/direct目录：

```
last.pth
train_log.json
experiment_config.json
learned_parameters.json
test_results/
  test_normal_metrics.json
  test_normal_predictions.csv
  test_normal_probabilities.pt
  test_fault_metrics.json
  test_fault_predictions.csv
  test_fault_probabilities.pt
  test_all_metrics.json
  test_all_predictions.csv
  test_all_probabilities.pt
completed.json
```

`completed.json`只在训练、三个split评估和必要文件均成功后写入，因此它是整个实验单元的提交标记。

4. Checkpoint内容

`last.pth`通常包含：

```
model_state
optimizer_state
epoch
config
extra metadata
```

不要仅根据文件名判断来源，应检查config/metadata中的participant、seed、scope、feature cache与 backbone checkpoint。

5. Feature cache

每个scope至少有训练和held-out测试cache。Direct Head Fusion复用这些cache，不重新提取RGB特征。metadata中的checkpoint路径可用于追溯representation。

6. 严格汇总目录

```
outputs/cross_person_summary_normal_only_ADJM_3seeds/  
outputs/cross_person_summary_all_runs_ADJM_3seeds/  
outputs/training_scope_comparison_ADJM_3seeds/  
outputs/direct_head_fusion_summary_ADJM_3seeds/
```

关键规模：

原十模型完整网格：
 $4 \times 3 \times 2 \times 10 \times 3 = 720$

原scope配对：
 $4 \times 3 \times 10 \times 3 = 360$

Direct：
 $4 \times 3 \times 2 \times 3 \times 3 = 216$

Direct对两个reference：
 $216 \times 2 = 432$

7. 防覆盖层级

Python层

`ensure_new_output_dir`拒绝非空目录，除非显式`--overwrite`。

脚本层

标准BAT/Slurm：

- completed存在则跳过；
- 目标非空但completed缺失则停止；
- 不传overwrite；
- 新实验写新目录。

汇总层

`--require-complete-grid`在写CSV之前检查网格。缺少任何model/split时停止，而不是用残缺数据生成看似完整的均值。

8. 中断恢复

若任务中断：

1. 先检查日志、目标目录和是否有completed；
2. 若completed存在，视为完成，不重跑；
3. 若目录非空但无completed，人工判断最后成功步骤；
4. 需要重跑时先把“该实验自己的不完整目录”改名备份；
5. 不删除或移动其他scope、其他seed或原M0-M6结果；
6. 重新运行标准入口。

9. 复现检查清单

- Python环境是否为预期PyTorch环境；

- dataset root与camera字段;
- Task Graph和relation matrix快照;
- held-out participant未进入训练;
- seed写入独立seed_N目录;
- normal/all backbone没有混用;
- feature metadata指向正确last checkpoint;
- M1-M6 baseline来自相同scope/seed M0;
- M6历史概率来自冻结M0, 不是真实标签;
- Direct入口没有baseline checkpoint;
- 固定epochs、无validation、使用last;
- 三个split全部完成;
- summary complete-grid通过。

10. 统计解释

正式总体:

先participant内平均3 seeds
→ 再对4 participants等权平均

不能把以下对象当作独立受试者:

- 12个participant-seed;
- 1895个clips;
- 同run内多个clips;
- 103个run跨participant的简单混合。

seed-fold正向计数与run正向计数是描述性稳定性证据。正式置信区间应使用participant/run层级或paired bootstrap, 并保留participant作为外层独立单位。

11. 任务边界

当前模型输出当前clip的node/Tier3类别。它没有直接输出:

- 是否漏做;
- 是否多做;
- 是否非法重复;
- 是否错误跳转;
- run级fault score。

fault split只是包含fault runs的clip分类测试集。因此“fault上的分类性能提高”不能写成“已实现fault detection”。

12. 部署边界

模型	部署性
M0	可部署，无历史
M1–M4	历史特征可获得时可部署
M5	不可部署，读取真实历史node
M6	可部署，使用M0 soft probabilities
Direct M1–M3	可部署，依赖同run历史feature
E2E	可部署，不依赖历史

真实在线部署还需要额外解决clip切分、历史缓存更新、run边界识别和延迟；当前离线实验默认这些信息已知。

来源文件: 08_configuration_assets_and_inventory.md

08 配置、资产、依赖与完整文件清单

1. configs/experiment_defaults.json

这是实验设计快照和默认参数说明，不是所有BAT/Slurm唯一读取源；实际运行值还可能由环境变量和命令行 覆盖。因此复现时必须同时保存最终experiment_config.json。

顶层任务配置

key	当前值	含义
test_participants	A,D,M	早期默认三折；严格扩展另含J
optional_strict_fourth_fold	J	后加入严格第四折
camera_id	001484412812	固定RGB相机
feature_dim	512	backbone fc前feature
num_nodes	35	graph node
num_tier3_classes	31	动作类别
n_frames	16	每clip输入帧
rgb_size	224	方形输入
rgb_mean/std	三个float	通道归一化

history

```
d_model=256
num_heads=4
max_history=35
dropout=0.1
epochs=50
batch_size=64
learning_rate=0.001
weight_decay=0.0001
action_loss_weight=0.0
```

action_loss_weight=0表示当前正式history模型只优化35-node交叉熵。

backbone

```
architecture=resnet3d_18
epochs=100
batch_size=16
learning_rate=0.0001
weight_decay=0.0001
checkpoint_policy=last_epoch_only
```

additional_e2e_baselines

记录normal-only增量E2E的100 epoch、batch 16、学习率/衰减和复用Tier3 last checkpoint策略。

complete_all_runs_pipeline

明确all-runs需要独立scratch backbone、独立features、M0–M6、三个E2E、三个测试split和matched normal-only比较。

recommended_strict_multiseed_extension

记录seed 1/2/42、A/D/J/M、J严格重训、无validation、last-only以及scope配对键。

direct_head_fusion_extension

```
models=m1_direct,m2_direct,m3_direct
scopes=normal_only,all_runs
reuse_existing_tier3_feature_caches=true
reuse_m0_checkpoint=false
uses_logit_delta=false
fusion_initialization=identity_current_zero_history
epochs=50
batch_size=64
learning_rate=0.001
weight_decay=0.0001
```

这些字段明确Direct实验与原delta模型隔离。

2. assets/integrated_task_graph_latest.json

这是35个可预测node的Task Graph快照。文档阅读时重点检查：

```
nodes
atomic_sequences
feature_history_constraints
execution_constraints
action_id_tier3
stage_id
```

代码要求1..35全部存在；缺少任意node会在TaskGraphSpec.load停止。

3. assets/integrated_feature_history_matrix.json

包含：

```
column_node_idx
rows[current_node_idx].values
```

行列顺序不假定天然为1..35，代码显式建立column_lookup再填[35, 35]。·会转成X。

4. assets/README.md

说明Task Graph与history matrix来源、方向、relation含义和快照使用约定。若未来替换资产，必须同时：

1. 更新两个JSON；
2. 保证node mapping一致；
3. 运行setup/smoke tests；
4. 使用新输出目录；
5. 不把不同graph版本结果直接合并。

5. requirements.txt

声明项目Python依赖。核心运行依赖包括：

- PyTorch与torchvision；
- NumPy；
- 视频/图像和数据处理相关库。

GPU版PyTorch必须与CUDA/驱动匹配，不能只依赖通用pip install命令。正式环境路径由Windows/HPC 配置指定。

6. 原包级说明文件

README.md

按时间记录：

- 初始A/D/M跨人实验；

- 增量E2E;
- 完整all-runs;
- 严格四折三seed;
- Direct Head Fusion;
- Dynamic Epoch Graph-Valid Shuffle。

因为它保留历史演化，早期章节中的默认三折或辅助all-runs不能覆盖后期严格定义。

EXPERIMENT_RESULTS_ANALYSIS_2026-07-24. md

正式结果解释报告。当前版本已扩展Direct结果。统计结论应追溯到outputs中的CSV/JSON和predictions。

COMPLETE_EXPERIMENT_CONFIGURATION. md

记录M0–M6、E2E、Direct与Dynamic实验的完整配置、运行方式、输出隔离和比较关系。它是当前统一的 配置/实验设计说明；本技术手册进一步解释实际类、函数、shape和调用链。

7. Python源码清单

graph_history/

文件	行数	职责
__init__.py	4	包初始化
constants.py	21	全局常量和关系ID
utils.py	175	I/O、seed、设备、checkpoint、安全目录
backbone.py	185	ResNet3D
data.py	281	RGB和history datasets/collate
graph.py	167	graph加载与graph-valid排序
protocols.py	144	LOSO协议
metrics.py	60	confusion与概率聚合
models.py	430	M0–M6和Direct
engine.py	217	feature模型训练评估
video_evaluation.py	203	E2E视频评估
dynamic_data.py	166	逐epoch、逐样本确定性graph-valid重排
dynamic_models.py	112	Dynamic Joint-Head Delta
dynamic_engine.py	84	epoch状态注入与dynamic训练循环

tools/

文件	行数	职责
validate_setup.py	52	环境检查
prepare_protocols.py	53	协议入口
guard_output_dir.py	24	防覆盖
train_backbone.py	163	Tier3训练
extract_features.py	108	feature cache
train_history_model.py	190	M0–M6
train_direct_history_model.py	206	Direct
evaluate_e2e_tier3.py	105	Tier3复测
train_e2e_node.py	205	E2E node
smoke_test_models.py	69	M1–M6 smoke
smoke_test_direct_models.py	78	Direct smoke
summarize_results.py	54	旧fold汇总
summarize_cross_person.py	182	早期跨人
summarize_all_models.py	583	正式十模型
summarize_direct_head_fusion.py	287	Direct汇总
train_dynamic_epoch_shuffle.py	308	三种Dynamic模型训练
summarize_dynamic_epoch_shuffle.py	379	Dynamic严格配对汇总
smoke_test_dynamic_epoch_shuffle.py	259	Dynamic数据、训练循环和模型回归测试

当前共14个包模块与18个Python工具入口；Dynamic实现独立成文件，原训练入口未改写。

8. 编排脚本清单

完整逐文件用途见第5卷。数量：

```
BAT                54
Slurm job scripts  38
HPC config/submit sh 17
```

Dynamic新增BAT 34–36、Slurm 36–38及独立组合/提交入口；原编号与调用链保持不变。

9. 结果文件不属于源码

outputs/包含大量：

```
.pth
.pt
.json
.csv
logs
completed markers
```

它们是实验证据，不应复制进源码文档或批量格式化。文档只读这些结果，新增文档不修改结果目录。

10. 当前文档覆盖表

内容	位置
研究问题与全流程	第0卷
所有关键数据字段/shape	第1卷
14个底层模块及函数	第2、3、9卷
18个Python入口及函数	第4、9卷
54个BAT	第5、9卷
38个Slurm作业	第5、9卷
17个shell配置/提交入口	第5、9卷
Direct新增结果	第6卷
Dynamic逐epoch重排新增阶段	第9卷
输出与防覆盖	第7卷
assets/config/requirements/原报告	本卷

来源文件：09_dynamic_epoch_shuffle.md

Dynamic Epoch Graph-Valid Shuffle实验记录

文档版本：2026-07-29 状态：实现与smoke test已完成；正式A/D/J/M × seeds 1/2/42结果待运行

1. 实验目的

原M3与M3 Direct在一个seed实验中为每个样本生成一次固定的graph-valid history顺序。新增阶段保持原 协议、特征、损失、测试split和静态实验不变，只把训练history改为“每个epoch重新生成一次合法顺序”，用于区分模型是否真正学习顺序扰动下稳定的历史信息。

2. 三个新增模型

模型	Node head初始化与训练	History分支	是否加载M0
m3_dynamic_frozen_m0_delta	从M0加载并冻结	只训练attention与delta head	是
m3_dynamic_joint_head_delta	随机初始化并训练	node head、attention与delta联合训练	否
m3_dynamic_direct_fusion	随机初始化并训练	特征融合后直接分类，不使用delta	否

特别注意：Joint-Head Delta不是“从M0解冻”。它从随机node head开始，因此可以单独检验联合训练相对 冻结M0基线的影响。

3. 重排与可复现性

训练样本的局部随机种子为：

```
SHA256(base_seed:epoch:sample_name)
```

因此同一base seed、epoch和样本会得到相同顺序，不同epoch通常产生不同合法顺序。排序仍调用原 graph-valid约束；如果历史长度不超过1，或重复node使合法候选只有一种，顺序保持不变。每次训练写出 shuffle_audit.json，记录多epoch出现多种顺序、相对actual变化和退化样本数量。

训练DataLoader强制persistent_workers=False，确保Windows多进程worker在每个epoch读取到更新后的 epoch状态。主测试仍用原M3的固定seeded graph-valid顺序，保证与静态M3和M3 Direct严格配对。

4. 实现边界

新增文件：

```
graph_history/dynamic_data.py
graph_history/dynamic_models.py
graph_history/dynamic_engine.py
tools/train_dynamic_epoch_shuffle.py
tools/summarize_dynamic_epoch_shuffle.py
tools/smoke_test_dynamic_epoch_shuffle.py
```

原data.py、models.py、engine.py及原训练入口不修改，旧实验仍走原逻辑。Dynamic只复用原有 graph-valid排序函数、M3 frozen-delta模型和M3 Direct模型。

5. 输出隔离与防覆盖

训练输出只写到：

```
outputs/<participant>_as_test/cam_001484412812/seed_<seed>/
history_models/dynamic_epoch_shuffle/<scope>/<model>/
```

汇总输出只写到：

```
outputs/dynamic_epoch_shuffle_summary_ADJM_3seeds/
```

训练目标目录非空即拒绝启动；已存在completed.json时，BAT/Slurm入口跳过已完成任务。不会写入原retrained_normal_only、retrained_all_runs、direct_head_fusion或E2E目录。

6. 运行方式

Windows单折单seed:

```
set TEST_PARTICIPANT=A
set SEED=1
call bat\run_dynamic_epoch_shuffle_one_fold.bat
```

Windows完整实验:

```
call bat\run_dynamic_epoch_shuffle_ADJM.bat
```

HPC单折单seed:

```
bash slurm/submit_dynamic_epoch_shuffle_one_fold.sh A 1 both
```

HPC完整实验:

```
bash slurm/submit_dynamic_epoch_shuffle_ADJM.sh
```

完整配置、超参数、配对基线和输出schema见包根目录 [COMPLETE_EXPERIMENT_CONFIGURATION.md](#) 第17节。